

CodeBounty — Complete Interview Prep Guide

0. First, the honest baseline (read this before anything else)

Based on your actual code, here is what is **genuinely built and working**:

- Clerk auth (sign in/up, GitHub OAuth) + middleware route protection
- User onboarding into Postgres via Prisma (`upsert` on first login)
- Role-based UI (ADMIN vs USER) in the navbar
- Admin problem creation form (React Hook Form + Zod) with Monaco code editor
- Judge0-backed **validation** of reference solutions before a problem is saved (multi-language: JS, Python, Java)
- Problem listing page with search, difficulty filter, tag filter, pagination, solved/unsolved indicator
- Code execution engine (`executeCode` server action) — submits to Judge0, polls results, compares actual vs expected output, stores a `Submission` + `TestCaseResult[]`, and upserts `ProblemSolved`
- Playlist creation + “add problem to playlist” flow
- Submission history fetch per user/problem
- Dockerized Postgres for local dev

What is **planned but not built** (this matters — see Section 9):

- Pattern system, Tags as relational entities, AI hints (Gemini), Sessions/Quizzes/Contests, pattern-progress heatmap, profile analytics page

Know this distinction cold. Interviewers will ask “is this live?” — never blur the line.

1. The 30-Second Elevator Pitch (use this when they say “tell me about a project”)

“I built CodeBounty, a coding practice platform similar to LeetCode, using Next.js, Prisma, PostgreSQL, and Clerk for auth. The core engineering challenge was building a secure code execution pipeline — I integrated Judge0 to compile and run user-submitted code in multiple languages, validate it against hidden test cases, and persist per-test-case results. I also built the admin problem-creation flow, which validates reference solutions against Judge0 *before* saving a problem to the database, so no problem with a broken solution ever gets published. Right now I’m extending it from a generic problem bank into a pattern-first learning platform — the schema groundwork for that is already sketched out.”

Practice saying this out loud until it's under 30 seconds without sounding memorized.

2. The 2-Minute Deep-Dive Version (use this after they say "go deeper")

Cover these beats in order:

1. **Problem it solves:** Most DSA platforms just dump a list of problems. Learners don't know *which pattern* a problem belongs to, so practice becomes random. CodeBounty's differentiator is organizing problems around solving patterns (sliding window, two pointers, DP, etc.) instead of a flat list — though right now the base "tutorial foundation" (auth, execution, submissions) is done and the pattern layer is the next milestone.
 2. **Stack + why:** Next.js App Router (server components + server actions cut down API boilerplate), Prisma + Postgres (relational data — users, problems, submissions, playlists all have real foreign-key relationships, so a relational DB made more sense than Mongo), Clerk (auth is a solved problem, don't reinvent session management/JWTs), Judge0 (open-source, sandboxed multi-language code execution — this is the same engine LeetCode-clone tutorials and some real platforms use).
 3. **The interesting engineering bit:** Code execution safety. You never execute user code directly on your server — Judge0 uses `isolate`, a sandbox that restricts CPU/memory/syscalls, so a malicious or infinite-loop submission can't take down your app server.
 4. **A real problem you hit:** Judge0 self-hosting on Windows (full story in Section 8).
 5. **Where it's going:** Pattern-based learning, hint ladder, hosted timed sessions — but be clear these are roadmap, not shipped.
-

3. Full Architecture Walkthrough

```
app/
  (auth)/                                → route group, no URL segment added
    sign-in/[[ ... sign-in]]/page.jsx
    sign-up/[[ ... sign-up]]/page.jsx
    layout.jsx                          → centers auth forms, no navbar
  (root)/
    layout.jsx                          → fetches currentUserRole(), renders Navbar + children
    page.jsx                            → landing page, calls onBoardUser() on every load
    problems/page.jsx                   → server component, fetches problems + current user, passes to client
api/
  create-problem/route.js               → Route Handler (POST), admin-only, validates via Judge0, saves Problem
  create-problem/page.jsx               → admin-only page rendering the create-problem form
  layout.js                             → root layout: ClerkProvider, ThemeProvider, fonts, Toaster

modules/
  auth/actions/index.js                 → server actions: onBoardUser, currentUserRole, getCurrentUser
  problems/actions/index.js             → server actions: getAllProblems, getProblemById, deleteProblem, exportProblems
  problems/components/                   → CreateProblemForm, ProblemsTable, CreatePlaylistModal, AddToPlaylistModal
  home/components/navbar.jsx            → role-aware navbar

lib/
  db.js                                 → Prisma client singleton (avoids connection exhaustion in dev hot reload)
  judge0.js                             → Judge0 API wrapper: language ID mapping, batch submit, batch poll
  utils.js                               → cn() helper for Tailwind class merging

prisma/
  schema.prisma
  migrations/                           → incremental migration history

middleware.js                           → clerkMiddleware, protects all routes except /sign-in, /sign-up
docker-compose.yml                       → local Postgres container
```

Why route groups `(auth)` and `(root)`? They let you have two completely different layouts (one with a navbar, one without) without adding `/auth` or `/root` to the URL. Pure organizational tool — no routing effect.

4. Database Schema — Full Breakdown

Models and what they represent

User

- `id` (uuid, PK), `clerkId` (unique — links to Clerk's identity), `email` (unique), `role` (enum: `ADMIN` | `USER`, default `USER`)
- Relations: owns `Problem[]` (as creator), `ProblemSolved[]`, `Submission[]`, `Playlist[]`
- Index on `role` — added later in a dedicated migration once role-based queries (admin checks) became frequent

Problem

- Core fields: `title`, `description`, `difficulty` (enum), `tags` (Postgres `String[]`), `constraints`, `hints`, `editorial`
- `examples`, `testCases`, `codeSnippets`, `referenceSolutions` are all stored as **JSON** columns
- `userId` → creator (FK to `User`, `onDelete: Cascade`)
- Index on `difficulty` (filter-heavy field)

Submission

- One row per code run/submit attempt: `sourceCode` (JSON), `language`, `stdin`, `stdout`, `stderr`, `compileOutput`, `status` ("Accepted"/"Wrong Answer"), `memory`, `time`
- FK to `User` and `Problem`, both cascade delete
- Has many `TestCaseResult`
- Index on `status`

TestCaseResult

- Per-test-case breakdown of a submission: `testCase` (index number), `passed` (bool), `stdout`, `expected`, `stderr`, `status`, `memory`, `time`
- FK to `Submission`, cascade delete, indexed on `submissionId`

ProblemSolved

- Join-style table tracking "this user solved this problem" — NOT every submission, just first success
- `@@unique([userId, problemId])` — this is what makes the `upsert` in `executeCode` idempotent (solving it twice doesn't create duplicates)

Playlist

- `name`, `description`, owned by `userId`
- `@@unique([name, userId])` — a user can't have two playlists with the same name, but different users can reuse names

ProblemInPlaylist

- Pure join table for the Playlist ↔ Problem many-to-many relationship
- `@@unique([playlistId, problemId])` prevents adding the same problem twice

Relationship diagram (verbal)

```
User 1---* Problem      (creator)
User 1---* Submission
User 1---* ProblemSolved
User 1---* Playlist
Problem 1---* Submission
Problem 1---* ProblemSolved
Problem 1---* ProblemInPlaylist
Submission 1---* TestCaseResult
Playlist 1---* ProblemInPlaylist
Playlist *---* Problem   (via ProblemInPlaylist – proper many-to-many)
```

Normalization — be ready to discuss this explicitly

This is a classic interview question (“did you normalize your schema? why/why not?”). Your honest, correct answer:

- **Playlist ↔ Problem is properly normalized (3NF)** via the `ProblemInPlaylist` join table with a composite unique constraint. This is the textbook-correct way to do many-to-many in a relational DB.
- **`Problem.tags` is a Postgres array column, not a normalized Tag table** — this is a deliberate denormalization. Why: tags in v1 are simple, low-cardinality labels used only for filtering; a full `Tag / ProblemTag` join adds complexity you don’t need yet. Postgres supports GIN indexes on array columns for fast `@>` / `&&` filtering, so it’s not even a real performance sacrifice.
- **`examples`, `testCases`, `codeSnippets`, `referenceSolutions` are JSON blobs, not relational tables.** This is intentional: each of these varies by language and has a nested, non-uniform shape (per-language input/output/explanation). Modeling this relationally would mean a `ProblemExample` table with a `language` discriminator column and a lot of joins for something that’s always read/written as one atomic unit anyway. JSON columns in Postgres (`jsonb` under the hood via Prisma’s `Json` type) are a legitimate, common pattern for “structured but variable” data — this isn’t a shortcut, it’s a documented tradeoff.
- **PRD roadmap explicitly plans to normalize tags** — `Pattern`, `Tag`, `ProblemTag` models are already sketched for Phase 2. This is actually a *good* answer to “what would you improve”: you already know the next step and why you didn’t do it yet (YAGNI — don’t build the join table until you actually need tag-level queries like “find all problems tagged both X and Y with counts”).

Indexing — talking points

- `Problem.difficulty` and `User.role` indexes were added in a **separate migration after the fact**, once those fields became the primary filter criteria in queries (`WHERE difficulty = ?` , `WHERE role = ?`). This shows you understand indexes should be added based on actual query patterns, not speculatively on every column.
- `Submission.status` is indexed for the same reason (filtering “Accepted” vs “Wrong Answer” in history views).

- `TestCaseResult.submissionId` is indexed because it's a foreign key that's always used in `WHERE submissionId = ?` lookups when rendering a submission's per-test-case breakdown.
- What you'd add next if asked "what index is missing": a composite index on `Submission(userId, problemId)` for faster "get my submissions for this problem" queries, and a GIN index on `Problem.tags` if tag filtering becomes a bottleneck.

Cascade deletes

Every child relation uses `onDelete: Cascade` (e.g., deleting a `User` deletes their `Submission`s, `ProblemSolved` records, `Playlist`s). Be ready to explain **why**: it keeps referential integrity — you never want orphaned submissions pointing at a deleted user. The tradeoff you should mention if pushed: cascade delete is convenient but dangerous at scale (a user-delete becomes a potentially large multi-table delete); a production system with millions of rows might instead soft-delete (`deletedAt` timestamp) or archive before hard-deleting.

Why Postgres over MongoDB?

Your data is inherently relational — users own problems, submissions reference both a user and a problem, playlists reference problems through a join table. Foreign keys, cascade behavior, and uniqueness constraints (like `@@unique([userId, problemId])`) are first-class relational concepts. Postgres also gives you `jsonb` columns, so you get the flexibility of a document store *where you actually need it* (examples, test cases) without giving up ACID transactions and joins everywhere else. That's a hybrid-strengths argument, which is a strong answer.

5. Core Workflows — Explain Each End-to-End

A. Auth + Onboarding

1. User signs in via Clerk (GitHub OAuth or email).
2. `middleware.js` runs `clerkMiddleware`, protects every route except `/sign-in` and `/sign-up` via `auth.protect()`.
3. On every load of the home page, `onBoardUser()` (a server action) runs `currentUser()` from Clerk, then `db.user.upsert()` — creates the user row on first login, updates name/email/image on subsequent ones. This keeps your DB in sync with Clerk without a webhook.
4. `currentUserRole()` fetches the role from your DB (not Clerk) — this is the source of truth for authorization, layered on top of Clerk's authentication.

Follow-up they might ask: "Why not just use Clerk's metadata for role instead of your own DB?" — Good answer: keeping role in your own relational DB lets you `JOIN` and filter on it in the same queries as everything else (e.g., `User_role_idx`), and decouples your authorization model from a third-party's schema.

B. Admin Problem Creation (the most “engineering-heavy” flow — know this cold)

1. Admin fills out the form (title, description, difficulty, tags, test cases, per-language starter code + reference solution + example).
2. Client-side validated with Zod (`problemSchema`) via `react-hook-form` + `zodResolver` .
3. POST to `/api/create-problem` (a Route Handler, not a server action — likely chosen here because it needed to be a distinct testable HTTP endpoint).
4. Route handler re-checks `currentUserRole() ≡ ADMIN` **server-side** — never trust the client, even though the UI already hides the button from non-admins. This is a genuinely good security practice to call out.
5. For **each language’s reference solution**, it maps the language to a Judge0 language ID (`getJudge0LanguageId`), builds one Judge0 submission per test case, and submits them as a **batch** (`submitBatch`) rather than one-by-one — this reduces round trips.
6. Polls Judge0 (`pollBatchResults`) until every submission in the batch is done (`status.id ≡ 1/2` , i.e., not “In Queue” or “Processing”).
7. If **any** test case fails for **any** language, the whole request fails with a 400 and the problem is **not saved**. Only if every language’s reference solution passes every test case does `db.problem.create()` run.

Why this matters as a talking point: this guarantees no problem in the database ever has a broken/incorrect reference solution — a real quality gate, not just a form submission.

C. Code Execution / Submission (`executeCode` server action)

1. User writes code in Monaco editor, clicks Run/Submit.
2. `executeCode(source_code, language_id, stdin[], expected_outputs[], problemId)` is called.
3. Validates `stdin` and `expected_outputs` arrays are same length and non-empty.
4. Builds one Judge0 submission per test case (`base64_encoded: false, wait: false` — async submission).
5. `submitBatch` → gets tokens → `pollBatchResults` → loops until all done.
6. Compares `result.stdout.trim()` against `expected_output.trim()` per test case → builds `detailedResults[]` , tracks `allPassed` .
7. Persists a `Submission` row with **stringified JSON arrays** for `stdout` / `stderr` / `memory` / `time` (one entry per test case) and `status: "Accepted" | "Wrong Answer"` .
8. If `allPassed` , **upserts** `ProblemSolved` — `upsert` here (not `create`) is deliberate: because of the `@@unique([userId, problemId])` constraint, re-solving an already-solved problem won’t throw a unique constraint error or create a duplicate row.
9. Bulk-inserts `TestCaseResult[]` via `createMany` , linked to the submission.
10. Refetches the submission **with its test cases included** and returns it to the client.

A genuinely good follow-up-proof detail: `stdout/stderr` are stored as JSON-stringified arrays on the `Submission` row itself, *and* individually on each `TestCaseResult` row. That’s some deliberate redundancy — the `Submission` -level fields are for a quick summary view, `TestCaseResult` rows are for the detailed per-test

breakdown. If asked “isn’t that duplicated data?” — yes, and that’s a reasonable interview answer: it’s a read-optimization tradeoff (avoid re-parsing/joining every time you just want a summary), at the cost of write-time duplication. This is a well-known pattern (denormalization for read performance).

D. Problem Listing

- Server component (`app/(root)/problems/page.jsx`) fetches the current DB user (for role) and calls `getAllProblems()` .
- `getAllProblems()` includes `solvedBy` filtered to the current user only (`where: { userId: data.id }`) — so each problem row cheaply tells you “did I solve this” without loading every user’s solve status.
- Client-side (`ProblemsTable`) does search/difficulty-filter/tag-filter/pagination **in-memory** using `useMemo` — fine at current scale, but you should be ready to say: “at scale, I’d move filtering/pagination server-side with query params, since loading every problem into the client doesn’t scale past a few hundred rows.”

E. Playlists

- `CreatePlaylistModal` → POST `/api/playlists` (not shown in provided files, but implied) → creates a `Playlist` row.
- `AddToPlaylistModal` → fetches user’s playlists, POST `/api/playlists/add-problem` → creates a `ProblemInPlaylist` row, protected by the unique constraint from being duplicated.

6. Judge0 Self-Hosting on Windows — Full Story (they WILL ask about a “hard problem you solved”)

Use the STAR format out loud.

Situation: Judge0 is the code execution engine I used in CodeBounty to run users’ submitted code securely. It uses the `isolate` sandbox to execute untrusted code while applying resource restrictions such as CPU time and memory limits. Since I was developing on Windows, I wanted to self-host Judge0 locally instead of relying entirely on the RapidAPI-hosted version, especially because frequent testing during development could be affected by API rate limits.

Task: I needed to get a reliable local Judge0 instance running for development and connect my CodeBounty application to it.

Problem (the actual technical issue): The main challenge was that Judge0’s sandboxing relies on Linux-specific functionality such as cgroups and namespaces. My development machine was Windows, so I initially tried running Judge0 through Docker Desktop with the WSL2 backend.

I ran into issues where:

- Some submissions remained stuck in the “**In Queue**” state (`status.id === 1`).

- The Judge0 worker encountered errors while trying to initialize the `isolate` sandbox.
- I faced permission and cgroup-related issues when the worker tried to access or configure the required Linux resources.

I spent time debugging the Docker, WSL2, cgroup, and worker configuration. I tried configuring the environment and adjusting the required permissions, but I wasn't able to get a reliable setup using Docker Desktop and WSL2 on Windows.

At that point, I realized that continuing to debug the Windows/WSL2 environment was adding unnecessary complexity. Since Judge0's sandboxing depends on Linux functionality, I decided to move the Judge0 environment to an actual Linux system.

Action: I created an Ubuntu Linux virtual machine using VirtualBox on my Windows machine.

Inside the Linux VM, I:

1. Installed Ubuntu.
2. Installed Docker.
3. Deployed Judge0 using Docker inside the Linux VM.
4. Configured the Judge0 containers and verified that code execution was working correctly.
5. Connected my CodeBounty application running on Windows to the Judge0 API running inside the Linux VM.

The final architecture looked like this:

```

Windows Development Machine
    |
    | VirtualBox
    v
Ubuntu Linux Virtual Machine
    |
    | Docker
    v
Judge0
    |
    v
Isolate Sandbox
    |
    v
User Code Execution
  
```

7. Other Challenges / Talking Points You Can Bring Up Proactively

Pick 2–3 of these to have fully rehearsed as “a challenge I faced” stories — interviewers love specificity over generic answers.

1. **Batch size limits on Judge0's batch endpoint:** Judge0's `/submissions/batch` endpoint has a max batch size (commonly 20). You handled this with a `chunkArray(arr, size = 20)` utility in `lib/judge0.js` — split large test-case sets into chunks before submitting. Good talking point on “designing around a third-party API's constraints.”
 2. **Polling vs webhooks:** Judge0 submissions are async — you poll `pollBatchResults` in a loop with a 1-second sleep until every submission's status leaves “In Queue”/“Processing.” Be ready for: “why not webhooks?” — Judge0 self-hosted does support callback URLs, but polling is simpler to reason about for a project at this scale and doesn't require exposing a public webhook endpoint during local dev.
 3. **Idempotency on “solve” tracking:** Using `upsert` with a composite unique constraint (`userId_problemId`) on `ProblemSolved` instead of `create` — this was a deliberate decision to avoid crashing on duplicate-key errors when a user re-solves a problem they already solved.
 4. **Prisma client singleton:** `lib/db.js` stores the Prisma client on `globalThis` in development to survive Next.js hot-reloads without exhausting your Postgres connection pool (a very common Prisma + Next.js gotcha — every hot reload would otherwise instantiate a new `PrismaClient`, and each one opens its own connection pool).
 5. **Judge0 language ID mapping:** You maintain your own map (`getJudge0LanguageId`) between your app's language names (`"PYTHON"`, `"JAVASCRIPT"`, etc.) and Judge0's numeric language IDs — a small but necessary translation layer between two systems' conventions.
 6. **Server-side re-validation of admin role:** Even though the UI hides “Create Problem”/“Delete” from non-admins, both the API route and the `deleteProblem` server action independently re-check `role === ADMIN` against the database. This is worth stating explicitly as “defense in depth” — client-side checks are UX, not security.
 7. **(Optional, only if pushed on code quality):** If an interviewer reads your code live and spots that `executeCode` assigns the `stderr` field twice in the `db.submission.create()` call — own it plainly: “yeah, that's leftover from an edit, functionally harmless since the second assignment wins, but I'd clean that up.” Confidently acknowledging a small real issue is *far* better than getting defensive or pretending not to see it.
-

8. Nextjs-Specific Questions You Should Expect (with your project as the

example)

Question	How to answer using YOUR code
App Router vs Pages Router — why App Router?	Server Components by default (less client JS), colocated layouts, native support for Server Actions which you use throughout <code>modules/*/actions</code> .
What's a Server Component vs Client Component here?	<code>app/(root)/problems/page.jsx</code> is a server component — it does <code>await currentUser()</code> and DB calls directly, no <code>useState / useEffect</code> . <code>ProblemsTable</code> , <code>CreateProblemForm</code> , <code>ModeToggle</code> are "use client" because they need interactivity (state, event handlers, hooks).
What are Server Actions and where do you use them?	Functions marked "use server" at the top of <code>modules/auth/actions/index.js</code> and <code>modules/problems/actions/index.js</code> — callable directly from client components like normal async functions, no manual <code>fetch</code> /API route needed. Next.js handles the RPC-style call under the hood.
Why is <code>/api/create-problem</code> a Route Handler instead of a Server Action?	It's called via <code>fetch()</code> from the client form rather than an imported function — Route Handlers make sense when you want a conventional REST-style endpoint (e.g., for external callers, or clearer separation of the heavy Judge0-validation logic). Server Actions and Route Handlers can coexist; you pick based on the calling pattern.
What are Route Groups — what's <code>(auth)</code> and <code>(root)</code> for?	Parentheses mean "don't add this to the URL." Lets <code>(auth)</code> and <code>(root)</code> each have their own <code>layout.jsx</code> (auth pages get a centered, nav-less layout; the rest of the app gets the navbar layout) without affecting routes like <code>/sign-in</code> or <code>/problems</code> .
Why is the middleware matcher written that way?	Excludes static assets/ <code>_next</code> internals by extension, but always runs on <code>/api</code> , <code>/trpc</code> , and Clerk's frontend API routes — standard Clerk middleware boilerplate to avoid protecting static files while still protecting actual pages/APIs.
What does <code>revalidatePath</code> do and where do you use it?	In <code>deleteProblem</code> , after deleting from the DB, <code>revalidatePath("/problems")</code> invalidates Next.js's cached render of that route so the next visit shows fresh data instead of a stale cached list.
Is this app statically generated or dynamic?	Dynamic — any page calling <code>currentUser()</code> (which reads cookies/headers) forces dynamic rendering per-request; you can't statically pre-render a personalized, auth-gated app like this.

Question	How to answer using YOUR code
How do you handle loading/error UI?	(Be honest about what you've actually implemented — <code>Spinner</code> and <code>Skeleton</code> components exist in your UI kit; mention if you've wired up <code>loading.js</code> / <code>error.js</code> conventions or not yet.)
Client-side state management — did you use Redux/Zustand?	No global state library; local component state (<code>useState</code>) plus server actions for data mutations is enough at this scale — Next.js's server-first model reduces the need for heavy client state management.
Why Tailwind v4 / shadcn (<code>base-ui</code>) instead of MUI/Chakra?	Utility-first CSS keeps styling colocated with markup, no runtime CSS-in-JS overhead; shadcn isn't a component <i>library</i> you install — it's components you own and can freely edit (visible in <code>components/ui/*</code>), avoiding the "fighting the library's API" problem.

9. Should You Mention the Future Scope / PRD Roadmap? **Yes — but frame it carefully.**

Do mention it. It shows product thinking beyond "I followed a tutorial," which is exactly what separates a project from a CRUD clone in an interviewer's eyes. But follow these rules:

1. **Always label it explicitly as "planned" / "roadmap" / "next milestone"** — never phrase it as if it's built.

E.g.:

"Right now problems are just a flat filterable list. The next phase — I've already modeled this in the schema — is a `Pattern` and `Tag` system so problems are organized by solving pattern (sliding window, two pointers, etc.) instead of a flat bank, plus a progressive AI hint ladder that won't just hand over the solution."

2. **Pick 1–2 roadmap items to go deep on, not all of them.** Good picks for a MERN/Next.js interview:

- **Pattern-first learning** (shows product/UX thinking)
- **The AI hint ladder** (shows you've thought about prompt design: Hint 1 = identify pattern → Hint 2 = key observation → Hint 3 = algorithm steps → Hint 4 = pseudocode → Hint 5 = complexity/mistakes — never full code immediately)
- Skip going deep on Sessions/Contests/Leaderboards unless directly asked — it's the least built-out part.

3. **If asked "why isn't X done yet?"** — don't get defensive. Answer with sequencing logic straight from your own plan: "I deliberately followed a 'foundation first' order — auth, execution engine, and submissions had

to be rock solid before I layered product differentiation on top, since those are the parts everything else depends on.”

4. **Never claim a percentage-complete number you can’t defend.** If pushed, describe it by milestone (“tutorial foundation is complete; pattern system and AI hints are next”) rather than “70% done.”
-

10. Full Anticipated Question Bank

Product / High-Level

- What problem does this solve that LeetCode doesn’t?
- Who is the target user?
- Walk me through the user journey from signup to solving a problem.
- What’s your data model for a “session”/“contest,” even though it’s not built? (Answer using PRD’s `Session`, `SessionProblem`, `SessionAttempt`, `SessionParticipant` models — explain the design: any user can host, not just “teachers,” ownership determines host actions.)
- What would you build next if you had one more week?

Database / Schema

- Walk me through your schema.
- Why Postgres over MongoDB for this?
- How do you handle many-to-many relationships? (Playlist↔Problem via join table.)
- What indexes did you add and why?
- Is your schema normalized? Where did you intentionally denormalize, and why?
- How do cascade deletes work here — what happens if you delete a User?
- Why JSON columns for examples/test cases instead of separate tables?
- How would you handle a Problem needing to support a 4th, 5th language?
- What’s a composite unique constraint and where do you use one? (`ProblemSolved`, `Playlist`, `ProblemInPlaylist`)
- How would you paginate at the database level instead of loading everything?
- How would you add full-text search on problem titles/descriptions?

Next.js / React

- App Router vs Pages Router.
- Server Components vs Client Components — how do you decide?
- What are Server Actions, and how are they different from a traditional REST API?
- How does Next.js middleware work, and what does yours protect?
- What’s hydration, and why does it matter for a Client Component like your Monaco editor?
- How would you add optimistic UI to the “solve” action?

- How would you cache the problems list, and how would you invalidate that cache when an admin adds a problem?
- What's the difference between `revalidatePath` and `revalidateTag` ?

Auth / Security

- Why Clerk instead of building auth yourself (or using NextAuth)?
- How do you know a user is who they say they are on the server, not just the client?
- Why do you re-check the admin role in the API route even though the UI already hides the button?
- How would you rate-limit hint requests or submission spam? (Reference the PRD's planned per-user rate limiting.)
- What happens to a user's data across the app if Clerk's account gets deleted? (Currently no webhook syncing that — good honest answer: "I haven't wired a Clerk deletion webhook yet; that's a gap I'd close before production.")

Code Execution / Judge0

- How does Judge0 actually run untrusted code safely?
- What's the difference between "Run" and "Submit" in a typical judge, and does your app distinguish them?
- Why batch submissions instead of one at a time?
- What happens if Judge0 times out or is down — how does your app degrade?
- How do you compare actual vs expected output — is it exact string match? What about trailing whitespace/newlines? (Your code does `.trim()` — mention the limitation: this won't catch semantically-equal-but-differently-formatted output, e.g., different float precision.)
- How would you support "custom input" / arbitrary user-provided test input (not just hidden test cases)?
- How would you scale code execution to handle 1000 concurrent submissions?

System Design / Scaling (be ready for at least a light version)

- How would this scale to 100k users?
- Where's your current bottleneck? (In-memory client-side filtering/pagination on the problems table is the honest answer — move to server-side query params + DB-level pagination.)
- How would you cache problem data (which changes rarely) vs submission data (which changes constantly)?
- How would you horizontally scale Judge0 workers?
- Would you introduce a message queue anywhere? (Submission processing is a natural fit — decouple "accept submission" from "execute and score it.")

Behavioral / Reflective

- What's the hardest bug you fixed in this project? (Use the Judge0/Windows story, or the batch-size chunking story.)
- What would you do differently if you started over?

- What's a design decision you're not 100% happy with? (Good honest answer: JSON blob duplication between `Submission` and `TestCaseResult`, or tags as a raw array instead of relational from day one.)
 - How did you decide what to build first? (Tutorial-foundation-first sequencing from your own build plan.)
-

11. Quick-Fire Cheat Sheet (memorize these one-liners)

- **App Router:** file-based routing where folders = routes, `page.jsx` = the route's UI, Server Components by default.
 - **Server Component:** renders on the server, can directly `await` a DB call, ships zero JS to the client for its own logic.
 - **Client Component:** opts in via `"use client"`, needed for hooks/state/event listeners/browser APIs.
 - **Server Action:** an async function marked `"use server"`, callable directly from a Client Component like an RPC — no manual fetch/JSON boilerplate.
 - **Middleware:** runs before a request reaches a route, used here for Clerk auth gating.
 - **Prisma singleton pattern:** store the client on `globalThis` in dev to prevent connection-pool exhaustion across hot reloads.
 - **Upsert:** create-if-absent, update-if-present — used for user onboarding and `ProblemSolved` idempotency.
 - **Cascade delete:** deleting a parent row auto-deletes dependent child rows to preserve referential integrity.
 - **Isolate (Judge0):** Linux sandbox using cgroups/namespaces to safely execute untrusted code with CPU/memory limits.
 - **Batch submission:** sending multiple Judge0 submissions in one request instead of N round trips, subject to a max batch size (you chunk at 20).
-

12. Final Delivery Tips

1. **Lead with the pitch, not the tech stack list.** Interviewers glaze over when you open with "I used Next.js, Tailwind, Prisma..." — open with the problem and the interesting engineering decision instead, then let the stack come up naturally when they ask "how did you build that."
2. **Always distinguish built vs planned** — this single habit builds more trust than anything else in this doc.
3. **When you don't know an answer, don't bluff.** Say "I haven't hit that yet, but here's how I'd approach it" and reason from first principles using what you *do* know about the schema/stack. Interviewers usually care more about your reasoning process than a memorized correct answer.
4. **Have the schema diagram (Section 4) mentally loaded** — many interviewers will literally ask you to draw or narrate your schema from memory. Practice narrating it without looking at this doc.
5. **Rehearse the Judge0/Windows story out loud at least 3 times** — it's your best "hard problem" answer because it's specific, technical, and has a clear before/after.
6. **Don't oversell the pattern/AI/sessions roadmap** — one confident, well-reasoned sentence about it is worth more than five sentences that make it sound half-built when it isn't built at all.

Good luck — you clearly understand this project at a deep level already; this is mostly about organizing what you know into a story you can deliver under pressure.